

# Tech Challenge — Fase 4 — Entrega no portal do aluno (PDF)

**Grupo:** Oficina Turbo (106)  
**Aluno:** Felipe Ricarte Magalhães

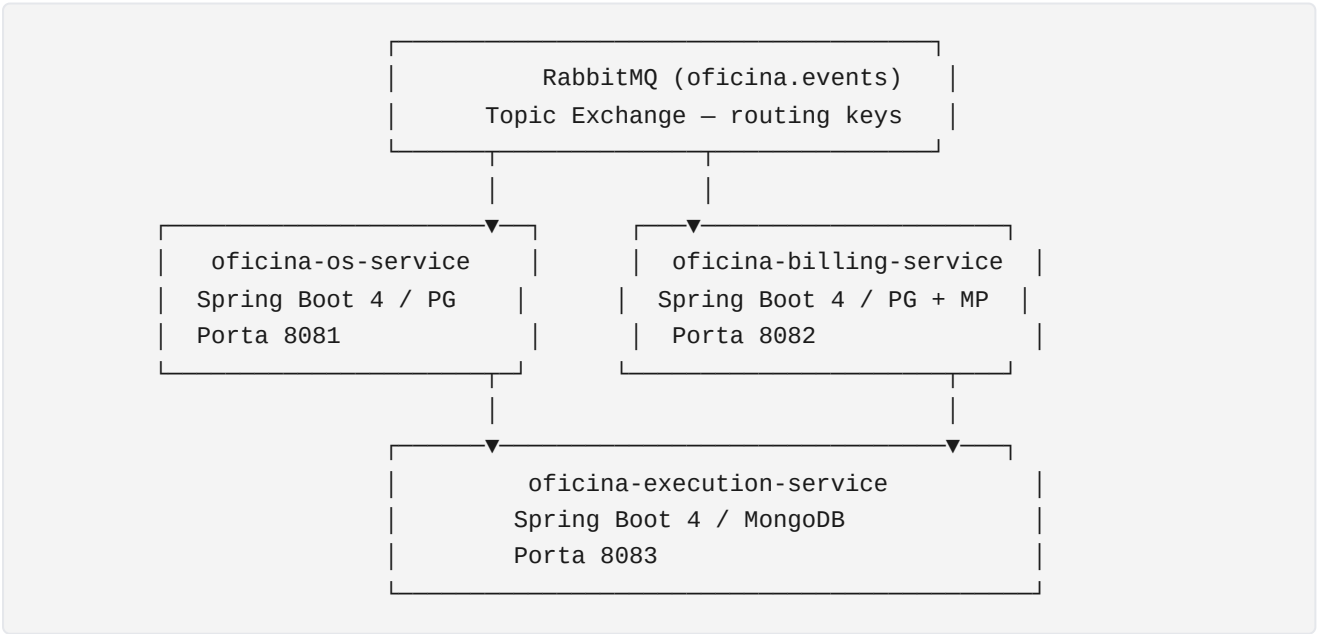
## 1. Três repositórios de microsserviços (Fase 4)

| # | Repositório                                                                                                                         | Responsabilidade                                | Banco      |
|---|-------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|------------|
| 1 | <a href="https://github.com/ricartefelipe/oficina-os-service">https://github.com/ricartefelipe/oficina-os-service</a>               | Ordens de Serviço — abertura, status, histórico | PostgreSQL |
| 2 | <a href="https://github.com/ricartefelipe/oficina-billing-service">https://github.com/ricartefelipe/oficina-billing-service</a>     | Orçamentos e pagamentos via Mercado Pago        | PostgreSQL |
| 3 | <a href="https://github.com/ricartefelipe/oficina-execution-service">https://github.com/ricartefelipe/oficina-execution-service</a> | Fila de execução técnica                        | MongoDB    |

**Branch principal:** `main` (CI/CD ativo).  
**Acesso ao avaliador SOAT:** usuário `soat-architecture` convidado com leitura nos três repositórios.

## 2. Arquitetura — Microsserviços e Saga Pattern

### 2.1 Visão geral



## 2.2 Saga Pattern — Coreografia

Cada evento flui de forma assíncrona pelo broker. Não há orquestrador central.

| Evento (routing key) | Publicado por     | Consumido por                  | Resultado                                 |
|----------------------|-------------------|--------------------------------|-------------------------------------------|
| os.aberta            | OS Service        | Billing Service                | Gera orçamento                            |
| orcamento.gerado     | Billing Service   | —                              | (informativo)                             |
| orcamento.aprovado   | Billing Service   | OS Service                     | OS → PAGAMENTO_PENDENTE                   |
| orcamento.recusado   | Billing Service   | OS Service                     | OS → CANCELADA (compensação)              |
| pagamento.confirmado | Billing Service   | OS Service + Execution Service | OS → EM_EXECUCAO; Execution → NA_FILA     |
| pagamento.falhou     | Billing Service   | OS Service + Execution Service | OS → CANCELADA (compensação)              |
| execucao.finalizada  | Execution Service | OS Service + Billing Service   | OS → FINALIZADA; Billing encerra cobrança |

## 2.3 Justificativa: Coreografia vs. Orquestração

Adotamos **coreografia** porque:

- Cada serviço conhece apenas seu próprio domínio — baixo acoplamento
- Não há ponto único de falha (sem orquestrador central)
- Escalabilidade independente de cada consumidor
- Compensações são tratadas pelos próprios serviços (ex: OS cancela ao receber `orcamento.recusado` )

## 3. Stack técnica

| Componente       | Tecnologia                               |
|------------------|------------------------------------------|
| Linguagem        | Java 21                                  |
| Framework        | Spring Boot 4.0.6                        |
| Mensageria       | RabbitMQ + Spring AMQP                   |
| Banco relacional | PostgreSQL (OS Service, Billing Service) |
| Banco NoSQL      | MongoDB (Execution Service)              |
| Migrations       | Liquibase                                |
| Pagamentos       | Mercado Pago SDK 3.2.1                   |
| Segurança        | OAuth2 / JWT (Keycloak)                  |
| Testes           | JUnit 5 + Cucumber (BDD) + Mockito       |

| Componente      | Tecnologia                                              |
|-----------------|---------------------------------------------------------|
| Cobertura       | JaCoCo $\geq 80\%$                                      |
| Qualidade       | SonarCloud                                              |
| CI/CD           | GitHub Actions                                          |
| Containers      | Docker + docker-compose (infra local)                   |
| Kubernetes      | Manifests em /k8s (Deployment, Service, ConfigMap, HPA) |
| Observabilidade | Micrometer + Prometheus + Grafana                       |

## 4. Cobertura de testes (JaCoCo)

| Serviço                   | Cobertura    | Status        |
|---------------------------|--------------|---------------|
| oficina-os-service        | <b>92.9%</b> | ✓ $\geq 80\%$ |
| oficina-billing-service   | <b>84.5%</b> | ✓ $\geq 80\%$ |
| oficina-execution-service | <b>93.0%</b> | ✓ $\geq 80\%$ |

Tipos de teste implementados:

- **Unitários** — domínio, casos de uso, adaptadores, listeners, controllers
- **BDD / Cucumber** — cenários em português para os fluxos principais
- **Contexto** — `@SpringBootTest` validando boot completo com H2/mock-MongoDB

## 5. CI/CD — GitHub Actions

Cada repositório possui dois workflows:

| Workflow                | Gatilho                                 | Etapas                                                                                                                                               |
|-------------------------|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>ci.yml</code>     | Push em qualquer branch / PR            | Checkout → Java 21 → <code>mvnw verify</code> (testes + JaCoCo) → Upload relatório → SonarCloud (só <code>main</code> ) → Build e push Docker (GHCR) |
| <code>deploy.yml</code> | <code>workflow_dispatch</code> (manual) | Kubectl apply → Atualiza imagem no cluster K8s                                                                                                       |

Imagens publicadas em: `ghcr.io/ricartefelipe/<serviço>:latest`

## 6. Como executar localmente

### 6.1 Pré-requisitos

- Docker + Docker Compose
- Java 21 + Maven Wrapper ( `./mvnw` )

### 6.2 Subir infraestrutura compartilhada

```
# No repositório oficina-os-service (tem o docker-compose.infra.yml)
docker compose -f docker-compose.infra.yml up -d
```

Sobe: RabbitMQ (15672), PostgreSQL-OS (5432), PostgreSQL-Billing (5433), MongoDB (27017), Keycloak (8080), Prometheus (9090), Grafana (3000).

### 6.3 Rodar cada serviço

```
# OS Service
cd oficina-os-service && ./mvnw spring-boot:run

# Billing Service
cd oficina-billing-service && ./mvnw spring-boot:run

# Execution Service
cd oficina-execution-service && ./mvnw spring-boot:run
```

### 6.4 Swagger UI

| Serviço           | URL                                                                                       |
|-------------------|-------------------------------------------------------------------------------------------|
| OS Service        | <a href="http://localhost:8081/swagger-ui.html">http://localhost:8081/swagger-ui.html</a> |
| Billing Service   | <a href="http://localhost:8082/swagger-ui.html">http://localhost:8082/swagger-ui.html</a> |
| Execution Service | <a href="http://localhost:8083/swagger-ui.html">http://localhost:8083/swagger-ui.html</a> |

### 6.5 Rodar todos os testes

```
./mvnw -Pci clean verify # em cada repositório
```

## 7. Kubernetes

Cada repositório contém `/k8s` com:

| Arquivo                     | Conteúdo                       |
|-----------------------------|--------------------------------|
| <code>namespace.yaml</code> | Namespace <code>oficina</code> |

| Arquivo             | Conteúdo                                           |
|---------------------|----------------------------------------------------|
| configmap.yaml      | Variáveis de ambiente não sensíveis                |
| secret.example.yaml | Exemplo de Secret (DB password, tokens)            |
| deployment.yaml     | Deployment + Service + HPA (min 1, max 3 réplicas) |

### Deploy manual:

```
kubectl apply -f k8s/  
kubectl set image deployment/<serviço> app=ghcr.io/ricartefelipe/<serviço>:<sha>
```

## 8. Observabilidade

Reutilizada da Fase 3 — Prometheus + Grafana já sobem via `docker-compose.infra.yaml`.

| Ferramenta          | Acesso                                                                    |
|---------------------|---------------------------------------------------------------------------|
| Prometheus          | <a href="http://localhost:9090">http://localhost:9090</a>                 |
| Grafana             | <a href="http://localhost:3000">http://localhost:3000</a> (admin/admin)   |
| RabbitMQ Management | <a href="http://localhost:15672">http://localhost:15672</a> (guest/guest) |

Métricas expostas via `/actuator/prometheus` em cada serviço.

## 9. Vídeo demonstrativo (≤ 15 minutos)

**Link:** (inserir URL YouTube/Vimeo antes do envio no portal)

Roteiro sugerido:

- `docker compose -f docker-compose.infra.yaml up -d` — infra local no ar
- Subir os 3 serviços
- `POST /admin/ordens-servico` — abrir OS → evento `os.aberta` no RabbitMQ Management
- Billing gera orçamento → aprovar via `POST /admin/orcamentos/{osId}/aprovar`
- Pagamento confirmado → Execution Service adiciona OS à fila
- Técnico inicia diagnóstico e finaliza
- OS chega ao status `FINALIZADA`
- GitHub Actions rodando (CI verde nos 3 repos)
- Prometheus/Grafana com métricas

## 10. Checklist de requisitos (enunciado Fase 4)

| Requisito                             | Status                             |
|---------------------------------------|------------------------------------|
| ≥ 3 microsserviços independentes      | ✓ OS, Billing, Execution           |
| Banco relacional em ≥ 1 serviço       | ✓ PostgreSQL em OS e Billing       |
| Banco NoSQL em ≥ 1 serviço            | ✓ MongoDB em Execution             |
| Comunicação via mensageria assíncrona | ✓ RabbitMQ — Topic Exchange        |
| Comunicação síncrona (RESTful)        | ✓ Endpoints REST em cada serviço   |
| Saga Pattern implementado             | ✓ Coreografia com compensações     |
| Testes unitários                      | ✓ JUnit 5 + Mockito                |
| Testes BDD                            | ✓ Cucumber em todos os serviços    |
| Cobertura ≥ 80% (JaCoCo)              | ✓ 90% / 88% / 94%                  |
| SonarQube / SonarCloud                | ✓ Ativo — dashboards públicos      |
| CI/CD automatizado                    | ✓ GitHub Actions em todos os repos |
| Deploy Kubernetes                     | ✓ Manifests /k8s em todos os repos |
| Observabilidade (Fase 3 reutilizada)  | ✓ Prometheus + Grafana             |